The session operator

The repo has no default system-prompt: each sub-project has its own, and a session picks one.

- **Local sessions** are started by a launcher (`scripts/claude-into-rtl-file--<option>.sh`, run by an IntelliJ run
  configuration). It passes `--setting-sources user` (so neither `CLAUDE.md` nor `.claude/settings.json` loads) and
  `--append-system-prompt-file <its prompt>` - a real system-prompt.
- **Sessions started without a launcher** - mainly cloud sessions (Claude Code on the web), and a plain `claude` in the
  repo root - are set up by this operator:
  - `.claude/settings.json` has a SessionStart hook that runs `operator.sh hook`. Its output goes into Claude's context
    before the user's first message: the environment, git status and a sync plan, instructions (cloud or local), the
    options, and (in cloud) the network.
  - `operator.sh choose <option> <session-id>` loads an option: Claude reads its prompt file in full and treats it as the
    system-prompt. The choice is recorded, and re-loaded by the hook after a compaction or a resume.
  - `recheck-after-git-sync.sh <session-id>` - after a git command that changed the files: what changed, and fresh reports.
  - `CLAUDE.md` is only a fallback, for when the hook did not run.
  - A session cannot start without a first message from the user (Claude cannot make the first move). So the first
    message is a shorthand: `start` (or a greeting, or `.`) = "show me the options"; `<option>: <task>` (e.g.
    `perush-hebrew: continue the commentary on Genesis 4`) = "load it, and do the task".

Files

File	What
`operator.sh`	The entry points: `hook`, `choose`, `status`, `options`, `network [--raw]`
`recheck-after-git-sync.sh`	The recheck after a sync
`_common.sh`	The shared code (sourced, not executed). macOS bash 3.2 compatible
`../../.claude/settings.json`	The SessionStart hook
`../../CLAUDE.md`	The fallback
`.git/operator-state/<session-id>.*`	Per-session state (never committed): `.option`, `.head`, `.branch`, `.boot`

The options are never hard-coded: each launcher carries metadata lines - `DESCRIPTION`, `EXTRA_CLAUDE_INSTRUCTIONS`, `NEEDED_DOMAINS` (`"ANY"` = the open web), `NEEDED_DOMAINS_FILE` - plus the `PROMPT_FILE` / `CLAUDE_SESSION_DIR` / `FIRST_PROMPT` lines it uses anyway. They are read as text, never executed (a launcher opens a Terminal window).
A new launcher is a new option - no change here.

What we know about cloud sessions

Found by a three-session experiment on 2026-10-21 (a new session, a second one, and the first one resumed after its VM was reclaimed). The operator relies on all of it.

The machine and the checkout

- A **new session** is a fresh VM (Ubuntu 24.04, user `root`, `/home/user/perush`) with a fresh **shallow clone** (50 commits) - not a worktree.
- The session starts on a cloud-chosen branch `claude/<adjective>-<name>-<id>`. The cloud also creates a local `main` tracking `origin/main`, and writes the session branch's `origin/claude/...` ref **locally** - the branch is **not on GitHub** (only `git ls-remote` tells the truth).

- An **idle session's VM is reclaimed** (here within ~30 minutes). **Resuming** it boots a new VM on the **same disk**:
  the checkout, its uncommitted work, unpushed commits, even `/tmp` - all as they were; uptime 0 minutes; and the checkout
  as old as its clone (in the experiment: 4 commits behind). It looks familiar - and it is stale. The operator records
  the `boot_id`, and says when the VM rebooted and when the checkout was cloned.
- The SessionStart hook runs on `startup`, `resume` and `compact` (a resume and a compaction in a row run it twice).
  On a resume it runs the **checkout's own** (maybe stale) operator - the recheck after the sync brings the new one.
- Installed: node 22, bun, python 3.13, perl, jq, gh, rg, curl, GNU coreutils, git-lfs. **Missing: gawk** (mawk only).
  The environment's **setup script** (claude.ai UI) runs as root before the session, cached for ~7 days.

Git

- Git goes through a GitHub proxy. **Pushing to `main` works.** Deleting a branch is refused (`git push --delete`).
- Commits are signed, committer `Claude <noreply@anthropic.com>`.
- The cloud's system prompt says to develop on the session branch and not to push elsewhere without explicit
  permission. The operator states that permission (the user wrote it): work on `main`.
- The cloud's **Stop hook** (`~/.claude/stop-hook-git-check.sh`) blocks the end of a turn when there are uncommitted
  changes or untracked files ("Please commit and push these changes to the remote branch"). It compares with
  `origin/<current branch>`, so working on `main` satisfies it.
- **Auto mode refuses** `git checkout -B` ("Irreversible Local Destruction") - and so, probably, `reset --hard` and
  `branch -f`. It allowed `checkout main`, `merge --ff-only`, `stash`, `rebase`, `stash pop --index`, a mixed `reset`,
  `branch -d`. The sync plans use only those. The plan for "something could be lost" (a backup branch, stash
  `--include-untracked`, rebase, `stash pop --index`) was run for real on a stale, messy session - and lost nothing.

The network

- All traffic goes through a local proxy (`HTTPS_PROXY=http://127.0.0.1:<port>`); `NO_PROXY` holds the package
  registries and the Anthropic API. There is no `HTTP_PROXY` (plain http is not proxied).
- The default network access lets in GitHub and the package registries; almost everything else is refused. The
  allowlist is set **only in the claude.ai UI** (the environment's settings -> Network access: "Limited" with "Allowed
  domains", or "Full"); nothing in the repo can change it.
- An allowed domain matches **that exact host only**: `www.sefaria.org` does not allow `sefaria.org` or `voices.sefaria.org`.
- A refused host: the proxy answers `CONNECT` with `403` (curl: `http_code` 000, `http_connect` 403, exit 56); its body
  says `request blocked: no rule or allowlist entry allows host "<host>"` - but curl does not show the body of a failed
  CONNECT. The proxy's status page (`$HTTPS_PROXY/__agentproxy/status`) lists recent refusals. The cloud may append an
  `[agent-proxy] ... connect_rejected` note to a Bash command's output.
- The session's **DNS does not resolve a blocked host** - a clean signal: CONNECT 403 + no DNS answer = the allowlist.
- **WebFetch** obeys the same allowlist, and reports a blocked host as `getaddrinfo ENOTFOUND <host>` - easy to mistake
  for a typo or a dead site. **WebSearch** works (it runs server-side) - its results may link to blocked hosts.
- An allowlist change reaches a **running** session's **proxy** within about a minute (curl, git) - but **not its DNS**,
  which stays as it was when the session was created (even across a reboot). So WebFetch sees an added host only in a
  new session. The operator reports such hosts as "reachable only through the proxy".
- MCP servers that need blocked hosts fail at the session's start (`ERR_PROXY_TUNNEL ... 403`).

Compaction

- After a compaction the cloud re-attaches only small files that were read before: it dropped the 24 KB perush
  introduction (which holds the dictionary). The operator's re-load says to re-read every file in full.
- An English line after the option's "speak Hebrew" made a session slip into English after a compaction: the re-load
  comes last in the hook's output, and starts with the option's language rule.

The hook's output

- SessionStart output over 10,000 characters reaches Claude only as a 2,000-character preview. The cloud hook output is
  ~5-7 KB, the network section compact (one line per kind of failure). Keep it so.
- Git in a fresh clone escapes non-ASCII file names (`core.quotePath`): the operator lists changed files with
  `-c core.quotePath=false`, so that Hebrew paths match.

Testing a change

- Locally: `echo '{"session_id":"T","source":"startup"}' | claude/operator/operator.sh hook` (add `CLAUDE_CODE_REMOTE=true`
  to see the cloud version). A fake proxy that refuses every CONNECT (a few lines of Python's `http.server`, run with
  `HTTPS_PROXY=http://127.0.0.1:<port>`) shows the blocked-network output. Git states (behind, diverged, dirty): a
  throwaway clone of a local bare repo.
- `getent` (the DNS check) and `/proc/sys/kernel/random/boot_id` exist only on Linux - on macOS those parts are skipped.
- Then a real cloud session: the hook's output is in its context; ask it to quote it.